

[Home](#) > [Tutorials](#) > [Python](#)

An Introduction to Hierarchical Clustering in Python

Understand the ins and outs of hierarchical clustering and its implementation in Python

Jan 19, 2023 · 17 min read



Zoumana Keita

A data scientist who likes to write and share knowledge with the data and IA community

TOPICS

Python

Machine Learning



Motivation

Imagine that you are a Data Scientist working for a retail company. Your boss requests that you segment customers into the following groups: low, average, medium, or platinum customers based on spending behavior for targeted marketing purposes and product recommendations.

Knowing that there is no such historical label associated with those customers, how is it possible to categorize them?

This is where clustering can help. It is an unsupervised machine learning technique used to group unlabeled data into similar categories.

This tutorial will focus more on the hierarchical clustering approach, one of the many techniques in unsupervised machine learning. It will start by providing an overview of what hierarchical clustering is, before comparing it to some existing techniques.

Then, it will walk you through a step-by-step implementation in Python using the popular Scipy library.



Definition of Hierarchical Clustering

A hierarchical clustering approach is based on the determination of successive clusters based on previously defined clusters. It's a technique aimed more toward grouping data into a tree of clusters called dendrograms, which graphically represents the hierarchical relationship between the underlying clusters.

Comparison of Hierarchical Clustering to Other Clustering Techniques

Hierarchical clustering is a powerful algorithm, but it is not the only one out there, and each type of clustering comes with its set of advantages and drawbacks.

Let's understand how it compares to other types of clustering, such as K-means and model-based clustering. Many more techniques exist, but these two, in addition to hierarchical clustering, are widely used and provide a framework to help better understand the others.

You can learn more about [clustering in machine learning](#) in our separate article, covering five essential clustering algorithms.

Hierarchical clustering vs K Means clustering

Unlike Hierarchical clustering, K-means clustering seeks to partition the original data points into "K" groups or clusters where the user specifies "K" in advance.

The general idea is to look for clusters that minimize the squared Euclidean distance of all the points from the centers over all attributes (variables or features) and merge those

TUTORIALS

categories

datacamp

EN

NEW YEAR
NEW SKILLS 50% OFF

Buy Now

- It is computationally efficient compared to hierarchical clustering and can be used to analyze large data sets.
- K-means is easier to understand and implement.

Drawbacks

- It is less flexible than hierarchical clustering because it constrains the user to specify the number of clusters beforehand, which may not be obvious in some situations.
- The result is not stable and changes from one iteration to another for the same data set.
- It is more sensitive to outliers because the use of outliers in the data impacts the mean of the cluster.
- Both k-means and hierarchical clustering are incapable of handling categorical data directly and may not work well with data that is not continuous or has very large variance.

Despite its limitations, k-means clustering is still a popular method due to its ease of use and computational efficiency. It is frequently used as a reference point for comparing the performance of other clustering techniques.

Model-based clustering

Both K-means and hierarchical clustering techniques use a distance matrix to represent the distances between all the points in the dataset. Model-based clustering, on the other hand, applies statistical techniques to identify clusters in the data. Below is the general process:

- Decide the statistical model to use and choose the number of clusters.
- Fit the model on the data.
- Identify the clusters based on the model parameters.

Benefits

- Model-based clustering is more flexible than hierarchical clustering because it allows the use of different models to identify different types of clusters.

- It works better on data with complex shapes or structures.

Drawbacks

- It's computationally more expensive than hierarchical clustering, especially for large data.
- It requires a better understanding of statistical modeling techniques since the choice of the model can affect the final result.
- Similarly to K-means clustering, it requires the number of clusters to be specified beforehand.

Applications of Hierarchical Clustering

Hierarchical clustering has a variety of applications in our day-to-day life, including (but by no means limited to) biology, image processing, marketing, economics, and social network analysis.

Biology

The clustering of DNA sequences is one of the biggest challenges in bioinformatics.

Biologists can leverage hierarchical clustering to study genetic relationships between organisms to classify those organisms into taxonomic groups. This is beneficial for quick analysis and visualization of the underlying relationships.

Image processing

Hierarchical clustering can be performed in image processing to group similar regions or pixels of an image in terms of color, intensity, or other features. This can be useful for further tasks such as image segmentation, image classification, and object recognition.

Marketing

Marketing specialists can use hierarchical clustering to draw a hierarchy between different types of customers based on their purchasing habits for better marketing strategies and product recommendations. For instance, different products in retails can be recommended to customers whether they are low, medium, or high spenders.

Social network analysis

Social networks are a great source of valuable information when exploited efficiently. Hierarchical clustering can be used to identify groups or communities and to understand their relationships to each other and the structure of the network as a whole.

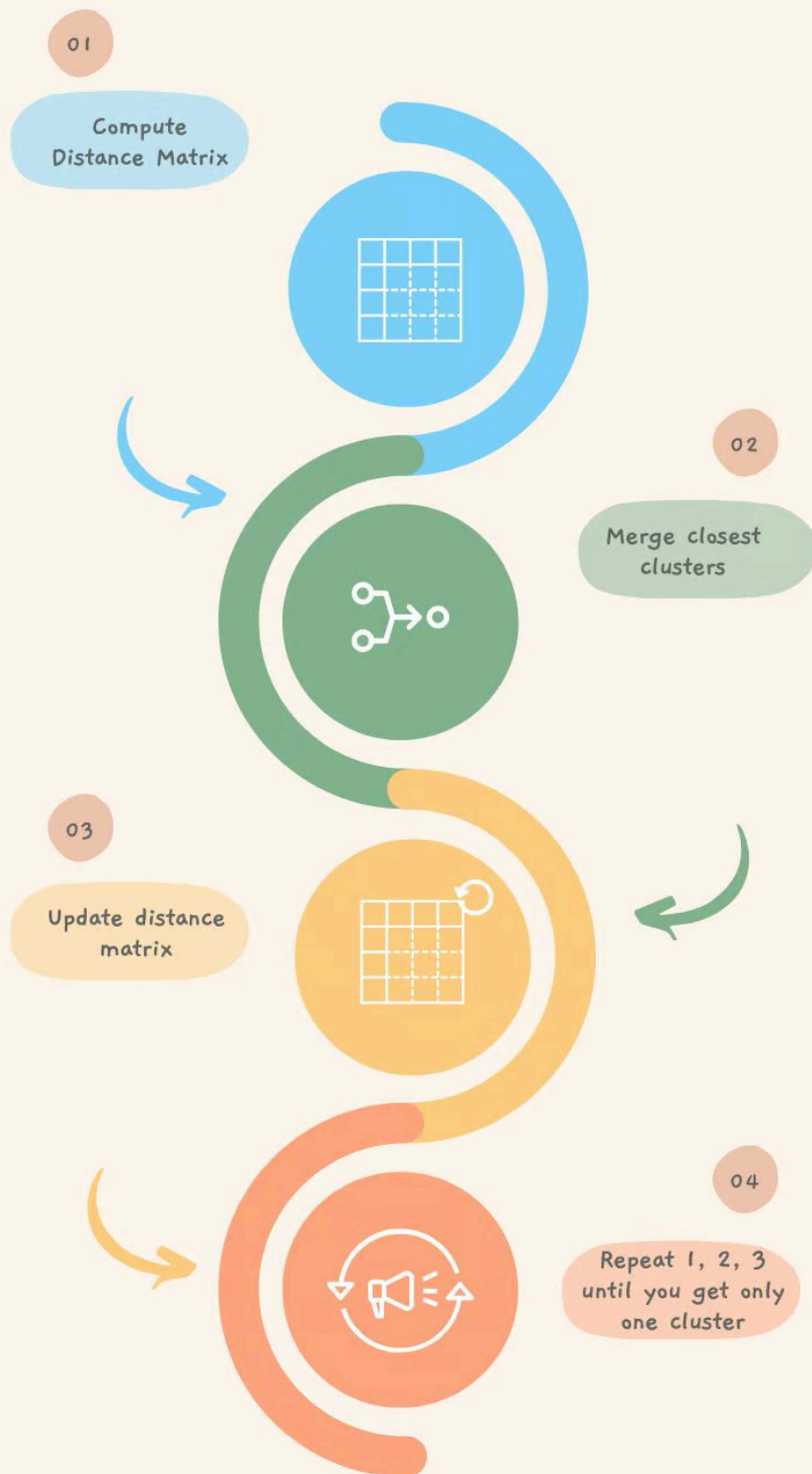
The Hierarchical Clustering Algorithm

In this section, we will look at three main concepts. The steps of the hierarchical algorithm, a highlight of the two types of hierarchical clustering (agglomerative and divisive), and finally, some techniques to choose the right distance measure.

Steps involved in the hierarchical clustering algorithm

The hierarchical clustering algorithm employs the use of distance measures to generate clusters. This generation process involves the following main steps:

Main Steps in the Hierarchical Clustering Algorithm



Preprocess the data by removing missing data and applying any additional tasks that make the data as clean as possible. This step is a more general one for most of the machine learning tasks.

1. Compute the distance matrix containing the distance between each pair of data points using a particular distance metric such as Euclidean distance, Manhattan distance, or cosine similarity. But the default distance metric is the Euclidean one.
2. Merge the two clusters that are the closest in distance.
3. Update the distance matrix with regard to the new clusters.
4. Repeat steps 1, 2, and 3 until all the clusters are merged together to create a single cluster.

Hierarchical clustering examples

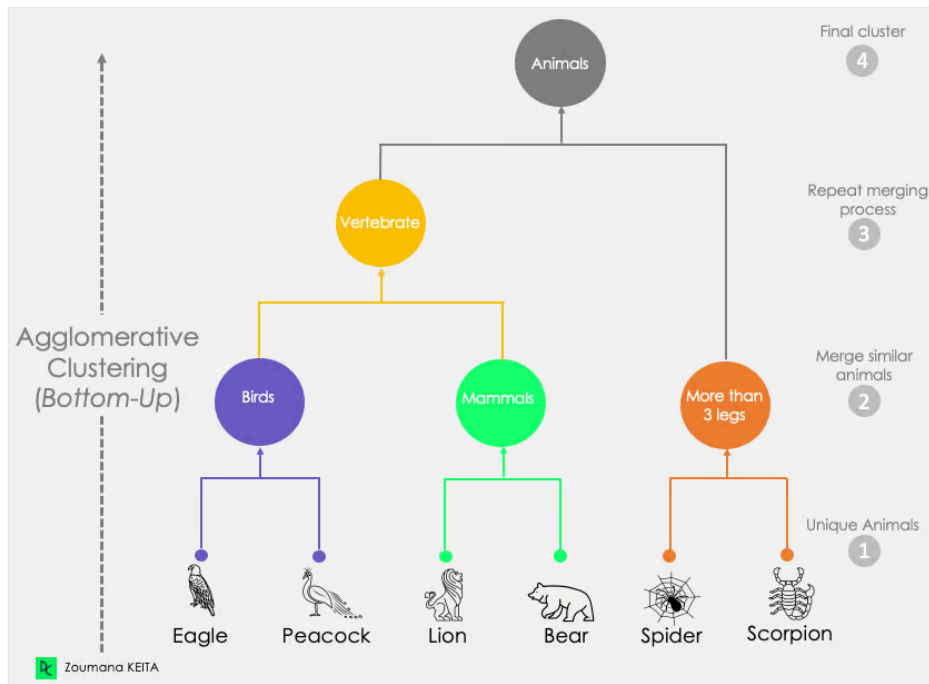
We can consider agglomerative and divisive clustering as mirrors of each other. Let's have a better look at how each one operates, along with a hierarchical clustering example and graphical visualization.

Agglomerative hierarchical clustering

This first scenario corresponds to the approach explained above. It starts by considering each observation as a singleton cluster (cluster with only one data point). Then iteratively merges clusters until only one cluster is obtained. This process is also known as the bottom-up approach.

As showing in the illustration below:

- We start by considering each animal to be its unique cluster.
- Then we generate three different clusters from those unique animals based on their similarities:
 - **Birds:** Eagle and Peacock
 - **Mammals:** Lion and Bear
 - **More than three leg animals:** Spider and Scorpion.
- We repeat the merging process to create the vertebrate cluster by combining the two most similar clusters: Birds and Mammals.
- After this step, the remaining two clusters, **Vertebrate** and **More than three legs**, are merged to create a single Animals cluster.



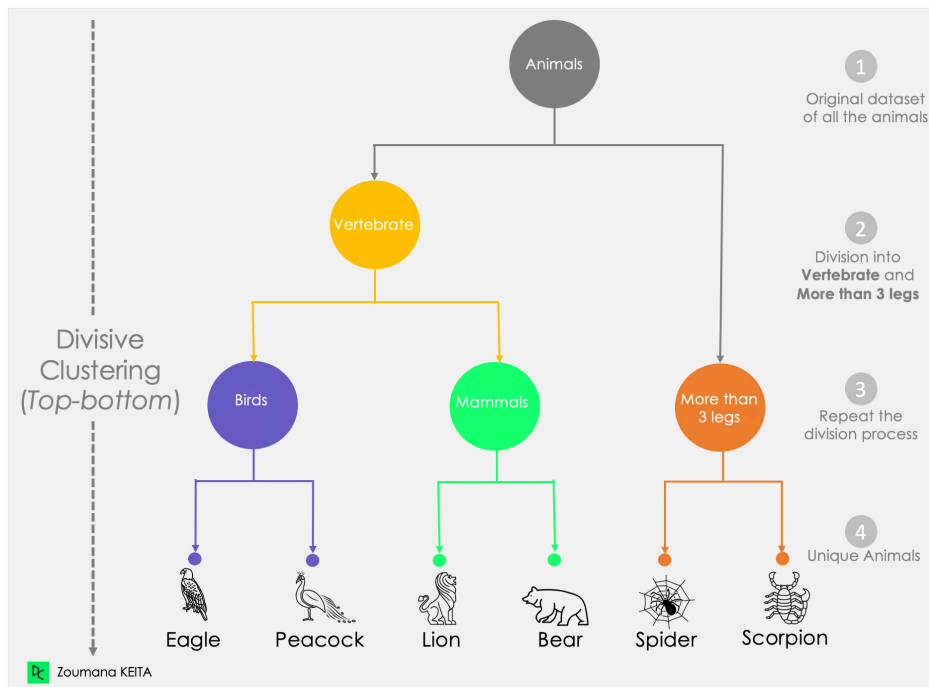
Dendrogram of Agglomerative Clustering Approach

Divisive clustering

On the other hand, divisive clustering is top-down because it starts by considering all the data points as a unique cluster. Then it separates them until all the data points are unique.

From this divisive approach graphic:

- We notice that the whole animal dataset is considered as a single bloc.
- Then, we divide that block into two clusters: Vertebrate and More than 3 legs.
- The division process is iteratively applied to the previously created clusters until we get unique animals.



Dendrogram of Divisive Clustering Approach

Choosing the right distance measure

Your choice of distance measure is a critical step in clustering, and it depends on the problem you're trying to solve. Considering the following scenario, we could cluster students based on any number of approaches such as their:

- Country of origin
- Gender, either male or female
- Previous academic background.

These are all valid clusters but differ in meaning.

Even though Euclidean distance is the most common distance used in most clustering software, other distance measures such as Manhattan distance, Canberra distance, Pearson or Spearman correlation, and Minkowski distance exist.

How to measure clusters before merging them

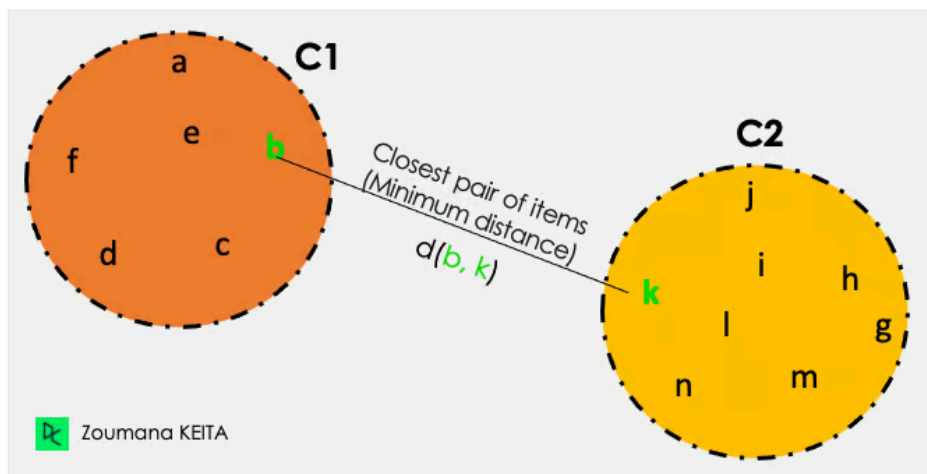
The previously mentioned distances are related to items. In this section, we will cover three standard ways (not exhaustive) to measure the nearest pair of clusters before merging them: (1) Single linkage, (2) Complete linkage, and (3) Average linkage.

Single linkage

From all the pairwise distances between the items in the two clusters C1 and C2, the single linkage takes the distance between the clusters as the minimum distance.

$$\text{Distance}(C1, C2) = \text{Min} \{ d(i, j), \text{ where item } i \text{ is within } C1, \text{ and item } j \text{ is within } C2 \}$$

Out of all the pairs of items from the two clusters, the ones highlighted in green have the minimum distance.



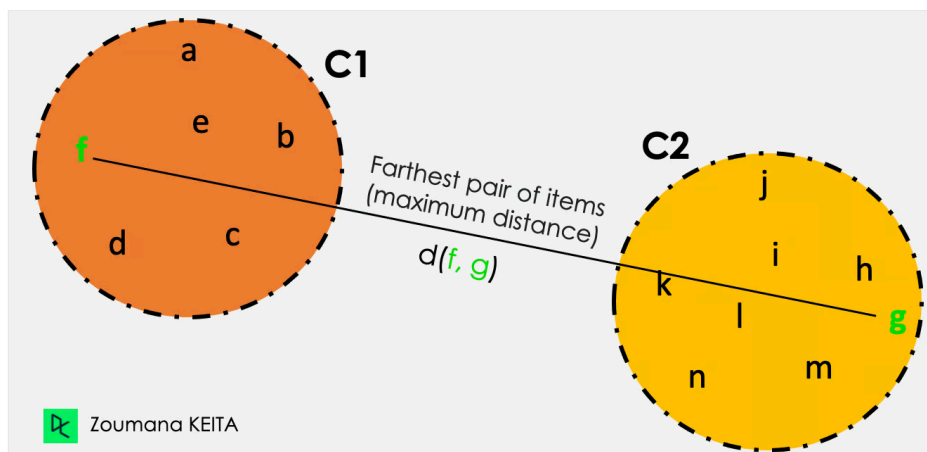
Single linkage illustration

Complete linkage

From all the pairwise distances between the items in the two clusters C1 and C2, the single linkage takes the distance between the clusters as the **maximum** distance.

$$\text{Distance}(C1, C2) = \text{Max} \{ d(i, j), \text{ where item } i \text{ is within } C1, \text{ and item } j \text{ is within } C2 \}$$

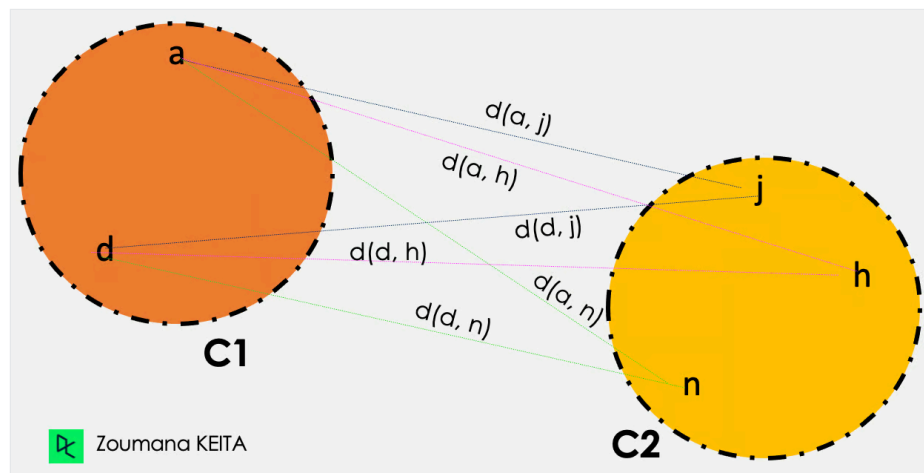
Out of all the pairs of items from the two clusters, the ones highlighted in green have the maximum distance.



*Complete linkage illustration***Average linkage**

In the average linkage clustering, the distance between two given clusters $C1$ and $C2$ corresponds to the average distances between all pairs of items in the two clusters.

$$\text{Distance}(C1, C2) = \text{Sum}\{d(i, j)\} / \text{Total Number of distances}$$

*Average linkage illustration*

Then the average linkage clustering is performed as follows

$$d(a, j) + d(a, h) + d(a, n) + d(d, j) + d(d, h) + d(d, n)$$

-----, where Total number of distances = 6

Total number of distances

Implementing Hierarchical Clustering in Python

Now you have an understanding of how hierarchical clustering works. In this section, we will focus on the technical implementation using Python.

If you are more interested in performing the implementation with R programming language, our [Hierarchical clustering in R tutorial](#) is where to start.

Setting up the environment

To begin, you will need to have [Python](#) installed on your computer, along with the following libraries:

- Pandas for loading the data frame.
- Scikit-learn to perform data normalization.
- Seaborn and matplotlib for visualization purposes.
- Scipy to apply the clustering.

You can install these libraries using pip, the Python package manager as follows:

```
pip install scikit-learn
pip install pandas

pip install matplotlib seaborn

pip install scipy
```

Explain code

POWERED BY datalab

Next, let's import the necessary modules and load the dataset. We will use the built-in iris dataset from scikit-learn, which contains information about different types of iris flowers.

To better illustrate the use case, we will be using the Loan Data available on [DataLab](#). All the code in this tutorial is available in [this DataLab workbook](#); you can easily create a copy of

your own to run the code in the browser, without having to install any software on your computer.

Understanding the data

The data set has 9,500 loans with information on the loan structure, the borrower, and whether the loan was paid back in full. We will get rid of the target column `not.fully.paid` to meet the unsupervised aspect.

```
import pandas as pd
loan_data = pd.read_csv("loan_data.csv")
loan_data.head()
```

 Explain code

POWERED BY  datalab

purpose	int.rate	installment	log.annual.inc	dti	fico	days.with.cr.line	revol.bal	revol.util	inq.last.6mths
debt_consolidation	0.1189	829.1	11.35040654	19.48	737	5639.958333	28854	52.1	
credit_card	0.1071	228.22	11.08214255	14.29	707	2760	33623	76.7	
debt_consolidation	0.1357	366.86	10.37349118	11.63	682	4710	3511	25.6	
debt_consolidation	0.1008	162.34	11.35040654	8.1	712	2699.958333	33667	73.2	
credit_card	0.1426	102.92	11.29973224	14.97	667	4066	4740	39.5	

First five rows of the load data

The following instruction shows that the data has 9,578 rows, and 14 columns of numerical types except for purpose, which is object and gives the textual description of the purpose of the load.

```
loan_data.info()
```

 Explain code

POWERED BY  datalab

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9578 entries, 0 to 9577
Data columns (total 14 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   credit.policy                         9578 non-null   int64
1   purpose                              9578 non-null   object
2   int.rate                             9578 non-null   float64
3   installment                           9578 non-null   float64
4   log.annual.inc                       9578 non-null   float64
5   dti                                   9578 non-null   float64
6   fico                                  9578 non-null   int64
7   days.with.cr.line                    9578 non-null   float64
8   revol.bal                             9578 non-null   int64
9   revol.util                           9578 non-null   float64
10  inq.last.6mths                       9578 non-null   int64
11  delinq.2yrs                          9578 non-null   int64
12  pub.rec                              9578 non-null   int64
13  not.fully.paid                       9578 non-null   int64
dtypes: float64(6), int64(7), object(1)
memory usage: 1.0+ MB
```

Information about the data

Preprocessing the data

Before the application of the clustering, the data need to be preprocessed to deal with missing information, normalize the column values, and drop irrelevant columns.

Deal with missing values

From the output below, we notice that there is no missing value in the data.

```
percent_missing = round(100*(loan_data.isnull().sum())/len(loan_data), 2)
percent_missing
```

[🔗 Explain code](#)POWERED BY  datalab

```
credit.policy      0.0
purpose            0.0
int.rate           0.0
installment        0.0
log.annual.inc     0.0
dti                0.0
fico               0.0
days.with.cr.line 0.0
revol.bal          0.0
revol.util         0.0
inq.last.6mths     0.0
delinq.2yrs        0.0
pub.rec            0.0
not.fully.paid     0.0
dtype: float64
```

Percentage of missing values in the data

Drop unwanted columns

We are going to analyze the loan data using all the columns except the following:

- Purpose
- Not.fully.paid, because it is the label telling whether the borrower made a full payment or not.

The cleaned_data corresponds to the data without the above columns.

```
cleaned_data = loan_data.drop(['purpose', 'not.fully.paid'], axis=1)
cleaned_data.info()
```

[🔗 Explain code](#)POWERED BY  datalab

The image below shows the information of the new data.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9578 entries, 0 to 9577
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   credit.policy          9578 non-null   int64
1   int.rate               9578 non-null   float64
2   installment            9578 non-null   float64
3   log.annual.inc         9578 non-null   float64
4   dti                    9578 non-null   float64
5   fico                   9578 non-null   int64
6   days.with.cr.line      9578 non-null   float64
7   revol.bal              9578 non-null   int64
8   revol.util             9578 non-null   float64
9   inq.last.6mths         9578 non-null   int64
10  delinq.2yrs            9578 non-null   int64
11  pub.rec                9578 non-null   int64
dtypes: float64(6), int64(6)
memory usage: 898.1 KB
```

New data without the unwanted columns

Outliers analysis

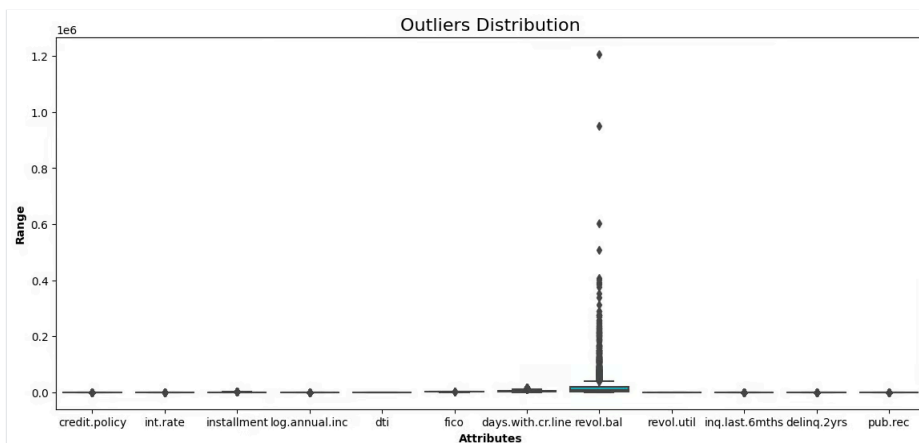
One of the weaknesses of hierarchical clustering is that it is sensitive to outliers. The distribution of each variable is given by the boxplot.

```
def show_boxplot(df):
    plt.rcParams['figure.figsize'] = [14,6]
    sns.boxplot(data = df, orient="v")
    plt.title("Outliers Distribution", fontsize = 16)
    plt.ylabel("Range", fontweight = 'bold')
    plt.xlabel("Attributes", fontweight = 'bold')

    show_boxplot(cleaned_data)
```

 Explain code

POWERED BY  datacamp



Boxplot of all the variables in the data

The borrower's revolving balance (revol_bal) is the only attribute with data points far away from the rest.

By using the interquartile range approach, we can remove all such points that lie outside the range defined by the quartiles $\pm 1.5 \times \text{IQR}$, where IQR is the InterQuartile Range.

This is achieved with the following helper function.

```
def remove_outliers(data):

    df = data.copy()

    for col in list(df.columns):

        Q1 = df[str(col)].quantile(0.05)
        Q3 = df[str(col)].quantile(0.95)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5*IQR
        upper_bound = Q3 + 1.5*IQR

        df = df[(df[str(col)] >= lower_bound) &

                (df[str(col)] <= upper_bound)]

    return df
```

 Explain code

POWERED BY  datalab

Then, we can apply the function to the data set.

```
without_outliers = remove_outliers(cleaned_data)
```

 Explain code

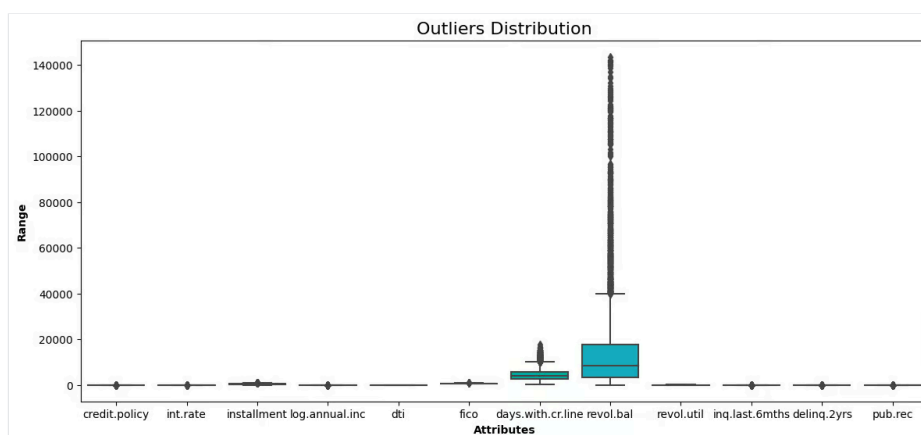
POWERED BY  datalab

Now we can check the new boxplot and compare it with the one before the removal of the outliers.

```
show_boxplot(without_outliers)
```

 Explain code

POWERED BY  datalab



No more data points lie outside the interquartile range.

```
without_outliers.shape
```

 Explain code

POWERED BY  datalab

The shape of the data is now 9,319 rows and 12 columns. This means that 259 observations were outliers, which have been dropped.

Rescale the data

Since hierarchical clustering uses Euclidean distance, which is very sensitive to dealing with variables with different scales, it's wise to rescale all the variables before computing the distance.

This is done using the `StandardScaler` class from `sklearn`.

```
from sklearn.preprocessing import StandardScaler

data_scaler = StandardScaler()

scaled_data = data_scaler.fit_transform(without_outliers)
scaled_data.shape
```

[🔗 Explain code](#)POWERED BY  datalab

The shape of the data remains the same (9,319 rows, 12 columns) because the normalization does not affect the same of the data.

Applying the hierarchical clustering algorithm

All the requirements are met to dive deep into the implementation of the clustering algorithm.

At this point, we can decide which linkage approach to adopt for the clustering in the `method` attribute of `linkage()` method. In this section, we will cover all three linkage techniques using the euclidean distance.

This is achieved with the codes below after importing the relevant libraries.

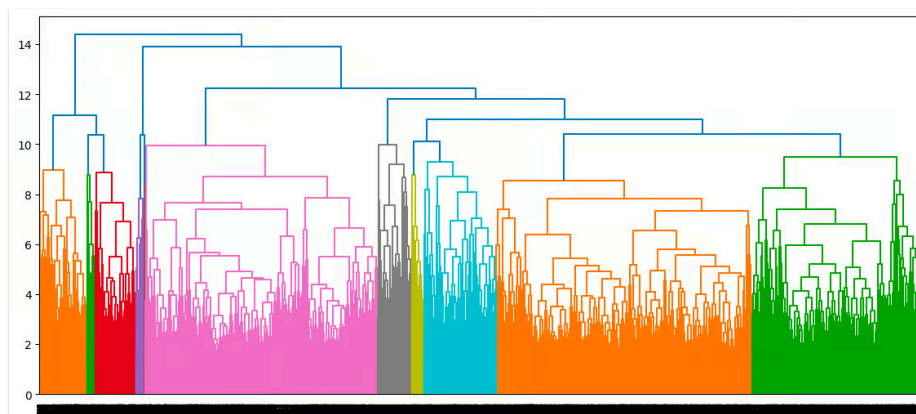
```
from scipy.cluster.hierarchy import linkage, dendrogram

complete_clustering = linkage(scaled_data, method="complete", metric="euclidean")
average_clustering = linkage(scaled_data, method="average", metric="euclidean")
single_clustering = linkage(scaled_data, method="single", metric="euclidean")
```

[🔗 Explain code](#)POWERED BY  datalab

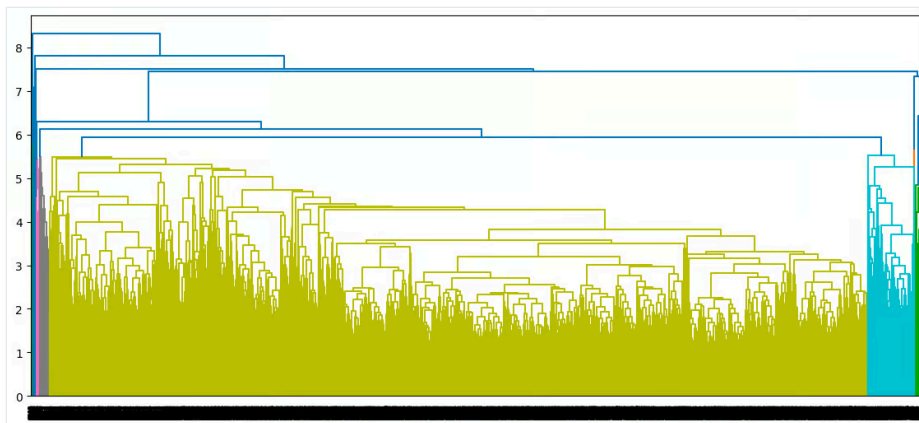
Once we have computed all the three clusterings, the corresponding dendrograms are shown as follows, starting with the complete clustering

```
dendrogram(complete_clustering)
plt.show()
```

[🔗 Explain code](#)POWERED BY  datalab

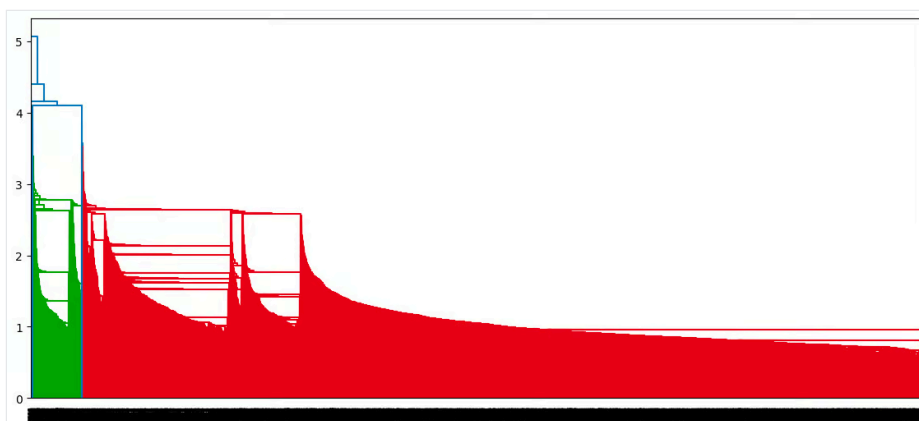
Dendrogram of the complete clustering approach

```
dendrogram(average_clustering)
plt.show()
```

[✎ Explain code](#)
POWERED BY  datalab

Dendrogram of the average clustering approach

```
dendrogram(single_clustering)
plt.show()
```

[✎ Explain code](#)
POWERED BY  datalab

Dendrogram of the single clustering approach

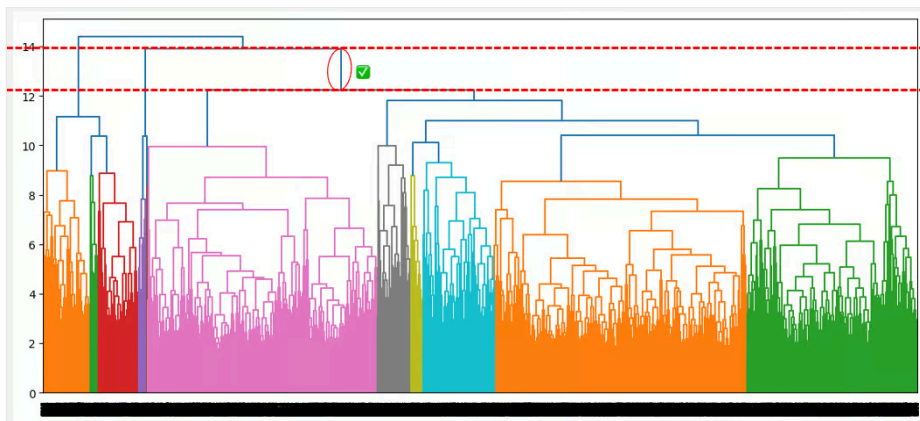
Interpreting the results (visualizing the dendrogram, determining the number of clusters)

For each linkage approach, how the dendrogram is constructed and every data point ends up into a single cluster.

- The x-axis of the dendrogram represents the samples in the data.
- The y-axis represents the distance between those samples. The higher the line, the more dissimilar are those samples/clusters.
- We get the appropriate number of clusters by drawing a horizontal line through that highest vertical line, and the number of intersections corresponds to the number of clusters.

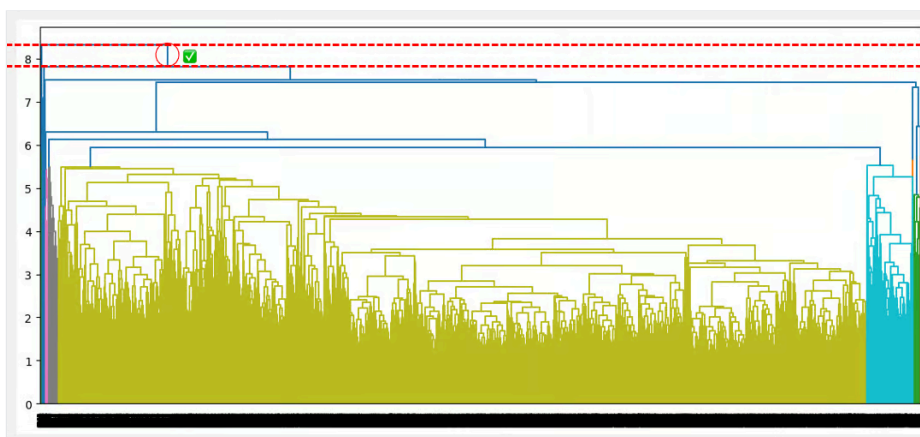
The optimal number of clusters can be obtained by identifying the highest vertical line that does not intersect with any other clusters (horizontal line). Such a line is found below with a red circle and green check mark.

- For complete linkage, it is the blue line on the right, and it generates three clusters.



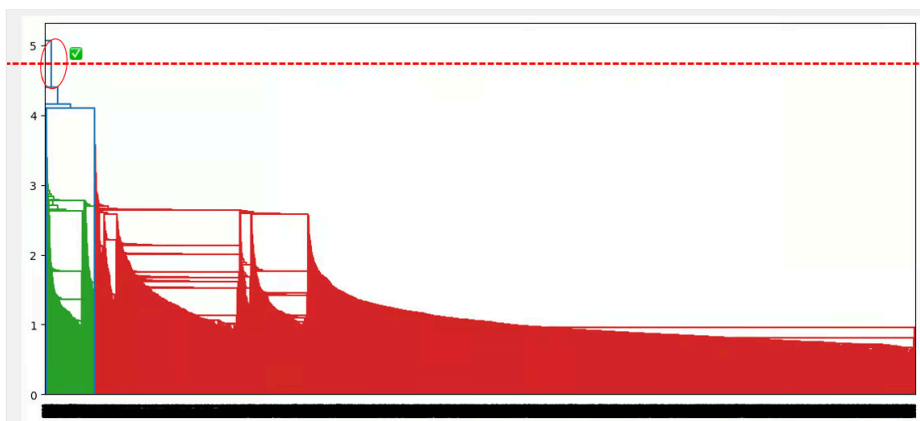
Optimal number of cluster from the highest distance without intersection (complete linkage)

- For the average linkage, it is the first blue vertical line, and it generates two clusters.



Optimal number of clusters from the highest distance without intersection (average linkage)

- For the single linkage, it is the first vertical line, and it generates only one cluster.



Optimal number of clusters from the highest distance without intersection (single linkage)

From the above observations, the average linkage seems to be the one that provides the best clustering, as opposed to the single and complete linkage, which respectively suggests considering one cluster and three clusters. Also, the optimal cluster number of two corresponds to our prior knowledge about the dataset, which is the two types of borrowers.

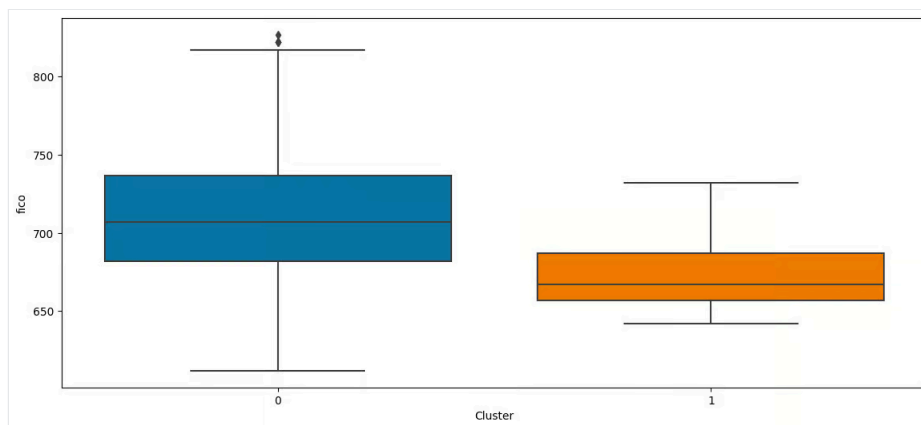
Now that we have found the optimal number of clusters let's look at what those clusters mean based on the borrower's credit score.

```
cluster_labels = cut_tree(average_clustering, n_clusters=2).reshape(-1, )
without_outliers["Cluster"] = cluster_labels

sns.boxplot(x='Cluster', y='fico', data=without_outliers)
```

[✎ Explain code](#)

POWERED BY  datacamp



From the above boxplot, we can observe that:

- Borrowers from cluster 0 have the highest credit scores.
- Whereas borrowers from cluster 1 have lower credit scores.

[Cluster Analysis in Python](#) can be a good next step to dive deep into K-means and hierarchical clustering using the Scipy library.

Conclusion

This article has covered what hierarchical clustering is, its benefits and weaknesses, and how it compares to k-means and model-based clustering.

We hope the article provides you with the relevant skills to efficiently cluster your unlabeled data for actionable decision-making.

Hierarchical Clustering FAQs

How do you choose the right number of clusters? ^

In hierarchical clustering, the right number of clusters can be determined from the dendrogram by identifying the highest distance vertical line which does not have any intersection with other clusters.

How do you handle categorical data in hierarchical clustering? v

How do you deal with large datasets? v

What is an agglomerative information bottleneck (AIB)? v

What is weighted hierarchical clustering? v

TOPICS

[Python](#)[Machine Learning](#)

Clustering Courses

COURSE

Cluster Analysis in Python

4 hr 55.1K

In this course, you will be introduced to unsupervised learning through techniques such as hierarchical and k-means clustering using the SciPy library.

[See Details →](#)[Start Course](#)[See More →](#)

Related

BLOG

Clustering in Machine Learning:
5 Essential Clustering...



TUTORIAL

Introduction to k-Means
Clustering with scikit-learn in...



TUTORIAL

Python Machine Learning:
Scikit-Learn Tutorial

[See More →](#)

Grow your data skills with DataCamp for Mobile

Make progress on the go with our mobile courses and daily 5-minute coding challenges.



LEARN

[Learn Python](#)[Learn R](#)[Learn AI](#)[Learn SQL](#)[Learn Power BI](#)[Learn Tableau](#)[Learn Data Engineering](#)[Assessments](#)[Career Tracks](#)[Skill Tracks](#)[Courses](#)[Data Science Roadmap](#)

DATA COURSES[Python Courses](#)[R Courses](#)[SQL Courses](#)[Power BI Courses](#)[Tableau Courses](#)[Alteryx Courses](#)[Azure Courses](#)[Google Sheets Courses](#)[AI Courses](#)[Data Analysis Courses](#)[Data Visualization Courses](#)[Machine Learning Courses](#)[Data Engineering Courses](#)[Probability & Statistics Courses](#)**DATALAB**[Get Started](#)[Pricing](#)[Security](#)[Documentation](#)**CERTIFICATION**[Certifications](#)[Data Scientist](#)[Data Analyst](#)[Data Engineer](#)[SQL Associate](#)[Power BI Data Analyst](#)[Tableau Certified Data Analyst](#)[Azure Fundamentals](#)[AI Fundamentals](#)**RESOURCES**[Resource Center](#)[Upcoming Events](#)[Blog](#)[Code-Alongs](#)[Tutorials](#)

[Docs](#)[Open Source](#)[RDocumentation](#)[Course Editor](#)[Book a Demo with DataCamp for Business](#)[Data Portfolio](#)[Portfolio Leaderboard](#)

PLANS

[Pricing](#)[For Business](#)[For Universities](#)[Discounts, Promos & Sales](#)[DataCamp Donates](#)

FOR BUSINESS

[Business Pricing](#)[Teams Plan](#)[Data & AI Unlimited Plan](#)[Customer Stories](#)[Partner Program](#)

ABOUT

[About Us](#)[Learner Stories](#)[Careers](#)[Become an Instructor](#)[Press](#)[Leadership](#)[Contact Us](#)[DataCamp Español](#)[DataCamp Português](#)[DataCamp Deutsch](#)[DataCamp Français](#)

SUPPORT

[Help Center](#)[Become an Affiliate](#)



[Privacy Policy](#) [Cookie Notice](#) [Do Not Sell My Personal Information](#) [Accessibility](#) [Security](#) [Terms of Use](#)

© 2025 DataCamp, Inc. All Rights Reserved.